Deducing this

Document #:	P0847R6
Date:	2021-01-15
Project:	Programming Language C++
Audience:	EWG

Reply-to: Gašper Ažman <gasper.azman@gmail.com>
Sy Brand <sibrand@microsoft.com>
Ben Deane, ben at elbeno dot com <ben@elbeno.com>
Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Abstract

We propose a new mechanism for specifying or deducing the value category of the expression that a member-function is invoked on. In other words, a way to tell from within a member function whether the expression it's invoked on is an lvalue or an rvalue; whether it is const or volatile; and the expression's type.

§ 2 Revision History

Changes since r5

Re-added section with the history of other syntaxes we considered (for posterity) and a discussion of reflection, explicit `static`, `virtual`, and coroutines. Rebased wording after Davis' paper. Re-worded so that explicit object member functions are *non-static* member functions rather than static member functions.

Changes since r4

Wording and Implementation. Discussion about implicit vs explicit invocation and interaction with static functions.

Changes since r3

The feedback from Belfast in EWG was "This looks good, come back with wording and implementation". This version adds wording, the implementation is in the works.

Changes since r2

[P0847R2] was presented in Kona in Jaunary 2019 to EWGI, with generally enthusiastic support.

This version adds:

— An FAQ entry for library implementor feedback

— An FAQ entry for implementability

— An FAQ entry for computed deduction, an orthogonal feature that EWGI asked for in Kona.

Changes since r1

[P0847R1] was presented in San Diego in November 2018 with a wide array of syntaxes and name lookup options. Discussion there revealed some potential issues with regards to lambdas that needed to be ironed out. This revision zeroes in on one specific syntax and name lookup semantic which solves all the use-cases.

Changes since r0

[P0847R0] was presented in Rapperswil in June 2018 using a syntax adjusted from the one used in that paper, using `this Self&& self` to indicate the explicit object parameter rather than the `Self&& this self` that appeared in r0 of our paper.

EWG strongly encouraged us to look in two new directions:

— a different syntax, placing the object parameter's type after the member function's parameter declarations (where the *cv-ref* qualifiers are today)

— a different name lookup scheme, which could prevent implicit/unqualified access from within new-style member functions that have an explicit self-type annotation, regardless of syntax.

This revision carefully explores both of these directions, presents different syntaxes and lookup schemes, and discusses in depth multiple use cases and how each syntax can or cannot address them.

§ 3 Motivation

In C++03, member functions could have *cv*-qualifications, so it was possible to have scenarios where a particular class would want both a `const` and non-`const` overload of a particular member. (Note that it was also possible to want `volatile` overloads, but those are less common and thus are not examined here.) In these cases, both overloads do the same thing — the only difference is in the types being accessed and used. This was handled by either duplicating the function while adjusting types and qualifications as necessary, or having one overload delegate to the other. An example of the latter can be found in Scott Meyers's "Effective C++" [EffCpp], Item 3:

```
class TextBlock {
public:
  char const& operator[](size_t position) const {
    // ...
    return text[position];
  }

  char& operator[](size_t position) {
    return const_cast<char&>(
      static_cast<TextBlock const&>(*this)[position]
    );
  }
  // ...
};
```

Arguably, neither duplication nor delegation via `const_cast` are great solutions, but they work.

In C++11, member functions acquired a new axis to specialize on: ref-qualifiers. Now, instead of potentially needing two overloads of a single member function, we might need four: `&`, `const&`, `&&`, or `const&&`. We have three approaches to deal with this:

— We implement the same member four times;

— We have three overloads delegate to the fourth; or

— We have all four overloads delegate to a helper in the form of a private static member function.

One example of the latter might be the overload set for `optional<T>::value()`, implemented as:

Quadruplication

```
template <typename T>
class optional {
  // ...
  constexpr T& value() & {
    if (has_value()) {
      return this->m_value;
    }
    throw
        bad_optional_access();
  }

  constexpr T const& value()
        const& {
    if (has_value()) {
      return this->m_value;
    }
    throw
        bad_optional_access();
  }

  constexpr T&& value() && {
    if (has_value()) {
      return move(this-
        >m_value);
    }
    throw
        bad_optional_access();
  }

  constexpr T const&&
  value() const&& {
    if (has_value()) {
      return move(this-
        >m_value);
    }
    throw
        bad_optional_access();
  }
  // ...
};
```

Delegation to 4th

```
template <typename T>
class optional {
  // ...
  constexpr T& value() & {
    return const_cast<T&>(
      static_cast<optional
        const&>(
        *this).value());
  }

  constexpr T const& value()
        const& {
    if (has_value()) {
      return this->m_value;
    }
    throw
        bad_optional_access();
  }

  constexpr T&& value() && {
    return const_cast<T&&>(
      static_cast<optional
        const&>(
        *this).value());
  }

  constexpr T const&&
  value() const&& {
    return static_cast<T
        const&&>(
      value());
  }
  // ...
};
```

Delegation to helper

```
template <typename T>
class optional {
  // ...
  constexpr T& value() & {
    return value_impl(*this);
  }

  constexpr T const& value()
        const& {
    return value_impl(*this);
  }

  constexpr T&& value() && {
    return
        value_impl(move(*this));
  }

  constexpr T const&&
  value() const&& {
    return
        value_impl(move(*this));
  }

private:
  template <typename Opt>
  static decltype(auto)
  value_impl(Opt&& opt) {
    if (!opt.has_value()) {
      throw
        bad_optional_access();
    }
    return forward<Opt>
        (opt).m_value;
  }
  // ...
};
```

This is far from a complicated function, but essentially repeating the same code four times — or using artificial delegation to avoid doing so — begs a rewrite. Unfortunately, it's impossible to improve; we *must* implement it this way. It seems we should be able to abstract away the qualifiers as we can for non-member functions, where we simply don't have this problem:

```
template <typename T>
class optional {
    // ...
    template <typename Opt>
    friend decltype(auto) value(Opt&& o) {
        if (o.has_value()) {
            return forward<Opt>(o).m_value;
        }
        throw bad_optional_access();
    }
```

```
    // ...
};
```

All four cases are now handled with just one function… except it's a non-member function, not a member function. Different semantics, different syntax, doesn't help.

There are many cases where we need two or four overloads of the same member function for different `const`- or ref-qualifiers. More than that, there are likely additional cases where a class should have four overloads of a particular member function but, due to developer laziness, doesn't. We think that there are enough such cases to merit a better solution than simply "write it, write it again, then write it two more times."

§ 4 Proposal

We propose a new way of declaring non-static member functions that will allow for deducing the type and value category of the class instance parameter while still being invocable with regular member function syntax. This is a strict extension to the language.

We believe that the ability to write *cv-ref qualifier*-aware member function templates without duplication will improve code maintainability, decrease the likelihood of bugs, and make fast, correct code easier to write.

The proposal is sufficiently general and orthogonal to allow for several new exciting features and design patterns for C++:

— recursive lambdas

— a new approach to mixins, a CRTP without the CRT

— move-or-copy-into-parameter support for member functions

— efficiency by avoiding double indirection with invocation

— perfect, sfinae-friendly call wrappers

These are explored in detail in the examples section.

This proposal assumes the existence of two library additions, though it does not propose them:

— `like_t`, a metafunction that applies the *cv*- and *ref*-qualifiers of the first type onto the second (e.g. `like_t<int&, double>` is `double&`, `like_t<X const&&, Y>` is `Y const&&`, etc.)

— `forward_like`, a version of `forward` that is intended to forward a variable not based on its own type but instead based on some other type. `forward_like<T>(u)` is short-hand for `forward<like_t<T,decltype(u)>>(u)`.

§ 4.1 Proposed Syntax

The proposed syntax in this paper is to use an explicit `this`-annotated parameter.

A non-static member function can be declared to take as its first parameter an *explicit object parameter*, denoted with the prefixed keyword `this`. Once we elevate the object parameter to a proper function parameter, it can be deduced following normal function template deduction rules:

```
struct X {
    void foo(this X const& self, int i);

    template <typename Self>
    void bar(this Self&& self);
};

struct D : X { };

void ex(X& x, D const& d) {
```

```
    x.foo(42);      // 'self' is bound to 'x', 'i' is 42
    x.bar();        // deduces Self as X&, calls X::bar<X&>
    move(x).bar();  // deduces Self as X, calls X::bar<X>

    d.foo(17);      // 'self' is bound to 'd'
    d.bar();        // deduces Self as D const&, calls X::bar<D const&>
}
```

Member functions with an explicit object parameter cannot be `static` or `virtual` and they cannot have *cv*- or *ref*-qualifiers. We will discuss the restriction on `static` and `virtual` in followup sections.

A call to a member function will interpret the object argument as the first (`this`-annotated) parameter to it; the first argument in the parenthesized expression list is then interpreted as the second parameter, and so forth.

Following normal deduction rules, the template parameter corresponding to the explicit object parameter can deduce to a type derived from the class in which the member function is declared, as in the example above for `d.bar()`).

We can use this syntax to implement `optional::value()` and `optional::operator->()` in just two functions instead of the current six:

```
template <typename T>
struct optional {
  template <typename Self>
  constexpr auto&& value(this Self&& self) {
    if (!self.has_value()) {
      throw bad_optional_access();
    }

    return forward<Self>(self).m_value;
  }

  template <typename Self>
  constexpr auto operator->(this Self&& self) {
    return addressof(self.m_value);
  }
};
```

This syntax can be used in lambdas as well, with the `this`-annotated parameter exposing a way to refer to the lambda itself in its body:

```
vector captured = {1, 2, 3, 4};
[captured](this auto&& self) -> decltype(auto) {
  return forward_like<decltype(self)>(captured);
}

[captured]<class Self>(this Self&& self) -> decltype(auto) {
  return forward_like<Self>(captured);
}
```

The lambdas can either move or copy from the capture, depending on whether the lambda is an lvalue or an rvalue.

§ 4.2 Proposed semantics

What follows is a description of how deducing `this` affects all important language constructs — name lookup, type deduction, overload resolution, and so forth.

§ 4.2.1 Name lookup: candidate functions

In C++17, name lookup includes both static and non-static member functions found by regular class lookup when invoking a named function or an operator, including the call operator, on an object of class type. Non-static member functions are

treated as if there were an implicit object parameter whose type is an lvalue or rvalue reference to *cv* `X` (where the reference and *cv* qualifiers are determined based on the function's own qualifiers) which binds to the object on which the function was invoked.

For non-static member functions using an explicit object parameter, lookup will work the same way as other member functions in C++17, with one exception: rather than implicitly determining the type of the object parameter based on the *cv*- and *ref*-qualifiers of the member function, these are now explicitly determined by the provided type of the explicit object parameter. The following examples illustrate this concept.

C++17	Proposed
`struct X {` `  // implicit object has type X&` `  void foo() &;` `  // implicit object has type X const&` `  void foo() const&;` `  // implicit object has type X&&` `  void bar() &&;` `};`	`struct X {` `  // explicit object has type X&` `  void foo(this X&);` `  // explicit object has type X const&` `  void foo(this X const&);` `  // explicit object has type X&&` `  void bar(this X&&);` `};`

Name lookup on an expression like `obj.foo()` in C++17 would find both overloads of `foo` in the first column, with the non-const overload discarded should `obj` be const.

With the proposed syntax, `obj.foo()` would continue to find both overloads of `foo`, with identical behaviour to C++17.

The only change in how we look up candidate functions is in the case of an explicit object parameter, where the argument list is shifted by one. The first listed parameter is bound to the object argument, and the second listed parameter corresponds to the first argument of the call expression.

This paper does not propose any changes to overload *resolution* but merely suggests extending the candidate set to include non-static member functions and member function templates written in a new syntax. Therefore, given a call to `x.foo()`, overload resolution would still select the first `foo()` overload if `x` is not `const` and the second if it is.

The behaviors of the two columns are exactly equivalent as proposed.

The only change as far as candidates are concerned is that the proposal allows for deduction of the object parameter, which is new for the language.

Since in some cases there are multiple ways to declare the same function, it would be ill-formed to declare two functions with the same parameters and the same qualifiers for the object parameter. This is:

```
struct X {
    void bar() &&;
    void bar(this X&&); // error: same this parameter type

    static void f();
    void f(this X const&); // error: two functions taking no parameters
};
```

But as long as any of the qualifiers are different, it is fine:

```
struct Y {
    void bar() &;
    void bar() const&;
    void bar(this Y&&);
};
```

The rule in question is 6.4.1 [basic.scope.scope]/3, and is extended in the wording below.

4.2.2 Type deduction

One of the main motivations of this proposal is to deduce the *cv*-qualifiers and value category of the class object, which requires that the explicit member object or type be deducible from the object on which the member function is invoked.

If the type of the object parameter is a template parameter, all of the usual template deduction rules apply as expected:

```
struct X {
  template <typename Self>
  void foo(this Self&&, int);
};

struct D : X { };

void ex(X& x, D& d) {
    x.foo(1);       // Self=X&
    move(x).foo(2); // Self=X
    d.foo(3);       // Self=D&
}
```

It's important to stress that deduction is able to deduce a derived type, which is extremely powerful. In the last line, regardless of syntax, `Self` deduces as `D&`. This has implications for name lookup within member functions, and leads to a potential template argument deduction extension.

4.2.3 By value `this`

But what if the explicit type does not have reference type? What should this mean?

```
struct less_than {
    template <typename T, typename U>
    bool operator()(this less_than, T const& lhs, U const& rhs) {
        return lhs < rhs;
    }
};

less_than{}(4, 5);
```

Clearly, the parameter specification should not lie, and the first parameter (`less_than{}`) is passed by value.

Following the proposed rules for candidate lookup, the call operator here would be a candidate, with the object parameter binding to the (empty) object and the other two parameters binding to the arguments. Having a value parameter is nothing new in the language at all — it has a clear and obvious meaning, but we've never been able to take an object parameter by value before. For cases in which this might be desirable, see by-value member functions.

4.2.4 Name lookup: within member functions

So far, we've only considered how member functions with explicit object parameters are found with name lookup and how they deduce that parameter. Now we move on to how the bodies of these functions actually behave.

Since the explicit object parameter is deduced from the object on which the function is called, this has the possible effect of deducing *derived* types. We must carefully consider how name lookup works in this context.

```
struct B {
    int i = 0;

    template <typename Self> auto&& f1(this Self&&) { return i;  }
    template <typename Self> auto&& f2(this Self&&) { return this->i; }
    template <typename Self> auto&& f3(this Self&&) { return forward_like<Self>(*this).i; }
```

```
    template <typename Self> auto&& f4(this Self&&) { return forward<Self>(*this).i; }
    template <typename Self> auto&& f5(this Self&& self) { return forward<Self>(self).i; }
};

struct D : B {
    // shadows B::i
    double i = 3.14;
};
```

The question is, what do each of these five functions do? Should any of them be ill-formed? What is the safest option?

We believe that there are three approaches to choose from:

1. If there is an explicit object parameter, `this` is inaccessible, and each access must be through `self`. There is no implicit lookup of members through `this`. This makes `f1` through `f4` ill-formed and only `f5` well-formed. However, while `B().f5()` returns a reference to `B::i`, `D().f5()` returns a reference to `D::i`, since `self` is a reference to `D`.

2. If there is an explicit object parameter, `this` is accessible and points to the base subobject. There is no implicit lookup of members; all access must be through `this` or `self` explicitly. This makes `f1` ill-formed. `f2` would be well-formed and always return a reference to `B::i`. Most importantly, `this` would be *dependent* if the explicit object parameter was deduced. `this->i` is always going to be an `int` but it could be either an `int` or an `int const` depending on whether the `B` object is const. `f3` would always be well-formed and would be the correct way to return a forwarding reference to `B::i`. `f4` would be well-formed when invoked on `B` but ill-formed if invoked on `D` because of the requested implicit downcast. As before, `f5` would be well-formed.

3. `this` is always accessible and points to the base subobject; we allow implicit lookup as in C++17. This is mostly the same as the previous choice, except that now `f1` is well-formed and exactly equivalent to `f2`.

Following discussion in San Diego, the option we are proposing is #1. This allows for the clearest model of what a `this`-annotated function is: it is a `static` member function that offers a more convenient function call syntax. There is no implicit `this` in such functions, the only mention of `this` would be the annotation on the object parameter. All member access must be done directly through the object parameter.

The consequence of such a choice is that we will need to defend against the object parameter being deduced to a derived type. To ensure that `f5()` above is always returning a reference to `B::i`, we would need to write one of the following:

```
template <typename Self>
auto&& f5(this Self&& self) {
    // explicitly cast self to the appropriately qualified B
    // note that we have to cast self, not self.i
    return static_cast<like_t<Self, B>&&>(self).i;

    // use the explicit subobject syntax. Note that this is always
    // an lvalue reference - not a forwarding reference
    return self.B::i;

    // use the explicit subobject syntax to get a forwarding reference
    return forward<Self>(self).B::i;
}
```

4.2.5 The Shadowing Mitigation / Private Inheritance Problem

The worst case for this proposal is the case where we do *not* intend on deducing a derived object - we only mean to deduce the qualifiers - but that derived type inherits from us privately and shadows one of our members:

```
class B {
    int i;
public:
    template <typename Self>
    auto&& get(this Self&& self) {
```

```
        // see above: we need to mitigate against shadowing
        return forward<Self>(self).B::i;
    }
};

class D : private B {
    double i;
public:
    using B::get;
};

D().get(); // error
```

In this example, `Self` deduces as `D` (not `B`), but our choice of shadowing mitigation will not work - we cannot actually access `B::i` from a `D` because that inheritance is private!

However, we don't have to rely on `D` to friend `B` to get this to work. There actually is a way to get this to work correctly and safely. C-style casts get a bad rap, but they are actually the solution here:

```
class B {
    int i;
public:
    template <typename Self>
    auto&& get(this Self&& self) {
        return ((like_t<Self, B>&&)self).i;
    }
};

class D : private B {
    double i;
public:
    using B::get;
};

D().get(); // now ok, and returns B::i
```

No access checking for the win.

4.2.6 Writing the function pointer types for such functions

As described in the previous section, the model for a member function with an explicit object parameter is a `static` member function.

In other words, given:

```
struct Y {
    int f(int, int) const&;
    int g(this Y const&, int, int);
};
```

While the type of `&Y::f` is `int(Y::*)(int, int) const&`, the type of `&Y::g` is `int(*)(Y const&, int, int)`. As these are *just* function pointers, the usage of these two member functions differs once we drop them to pointers:

```
Y y;
y.f(1, 2); // ok as usual
y.g(3, 4); // ok, this paper

auto pf = &Y::f;
pf(y, 1, 2);              // error: pointers to member functions are not callable
(y.*pf)(1, 2);            // okay, same as above
std::invoke(pf, y, 1, 2); // ok
```

```
auto pg = &Y::g;
pg(y, 3, 4);              // okay, same as above
(y.*pg)(3, 4);            // error: pg is not a pointer to member function
std::invoke(pg, y, 3, 4); // ok
```

The rules are the same when deduction kicks in:

```
struct B {
    template <typename Self>
    void foo(this Self&&);
};

struct D : B { };
```

Types are as follows:

— Type of `&B::foo<B>` is `void(*)(B&&)`

— Type of `&B::foo<B const&>` is `void(*)(B const&)`

— Type of `&D::foo<B>` is `void(*)(B&&)`

— Type of `&B::foo<D>` is `void(*)(D&&)`

This is exactly what happens if `foo` is a normal function.

By-value object parameters give you pointers to function in just the same way, the only difference being that the first parameter being a value parameter instead of a reference parameter:

```
template <typename T>
struct less_than {
    bool operator()(this less_than, T const&, T const&);
};
```

The type of `&less_than<int>::operator()` is `bool(*)(less_than<int>, int const&, int const&)` and follows the usual rules of invocation:

```
less_than<int> lt;
auto p = &less_than<int>::operator();

lt(1, 2);            // ok
p(lt, 1, 2);         // ok
(lt.*p)(1, 2);       // error: p is not a pointer to member function
invoke(p, lt, 1, 2); // ok
```

§ 4.2.7 Pathological cases

It is important to mention the pathological cases. First, what happens if `D` is incomplete but becomes valid later?

```
struct D;
struct B {
    void foo(this D const&);
};
struct D : B { };
```

Following the precedent of [P0929R2], we think this should be fine, albeit strange. If `D` is incomplete, we simply postpone checking until the point where we actually need a complete type, as usual. At that point `D().foo()` would be a valid expression. We see no reason to reject.

For unrelated complete classes or non-classes:

```
struct A { };
struct B {
```

```
    void foo(this A&);
    void bar(this int);
};
```

These are even more unlikely to be actually useful code. In this example, `B` is neither convertible to `A` nor `int`, so neither of these functions is even invocable using normal member syntax. However, you could take a pointer to such functions and invoke them through that pointer. `(&B::bar)(42)` is a valid, if weird, call.

We think these declarations can best be left for compilers to warn about if they so choose, rather than coming up with a language rule to reject them.

Another interesting case, courtesy of Jens Maurer:

```
struct D;
struct B {
  int f1(this D);
};
struct D1 : B { };
struct D2 : B { };
struct D : D1, D2 { };

int x = D().f1();  // error: ambiguous lookup
int y = B().f1();  // error: B is not implicitly convertible to D
auto z = &B::f1;   // ok
z(D());            // ok
```

Even though both `D().f1()` and `B().f1()` are ill-formed, for entirely different reasons, taking a pointer to `&B::f1` is acceptable — its type is `int(*)(D)` — and that function pointer can be invoked with a `D`. Actually invoking this function does not require any further name lookup or conversion because by-value member functions do not have an implicit object parameter in this syntax (see by-value `this`). The same reasoning holds for the direct function invocation.

Again, we're not sure if these formulations are actually useful. More so that they don't seem harmful and attempting to reject these cases may accidentally reject useful ones.

4.2.8 Teachability Implications

Explicitly naming the object as the `this`-designated first parameter fits within many programmers' mental models of the `this` pointer being the first parameter to member functions "under the hood" and is comparable to its usage in other languages, e.g. Python and Rust. It also works as a more obvious way to teach how `std::bind`, `std::thread`, `std::function`, and others work with a member function pointer by making the pointer explicit.

As such, we do not believe there to be any teachability problems.

4.2.9 Can `static` member functions have an explicit object type?

No. Static member functions currently do not have an implicit object parameter, and therefore have no reason to provide an explicit one.

4.2.10 Interplays with capturing `[this]` and `[*this]` in lambdas

Interoperability is perfect, since they do not impact the meaning of `this` in a function body. The introduced identifier `self` can then be used to refer to the lambda instance from the body.

4.2.11 Parsing issues

The proposed syntax has no parsings issue that we are aware of.

4.2.12 Code issues

There are two programmatic issues with this proposal that we are aware of:

1. Inadvertently referencing a shadowing member of a derived object in a base class `this`-annotated member function. There are some use cases where we would want to do this on purpose (see crtp), but for other use-cases the programmer will have to be aware of potential issues and defend against them in a somewhat verbose way.

2. Because there is no way to *just* deduce `const` vs non-`const`, the only way to deduce the value category would be to take a forwarding reference. This means that potentially we create four instantiations when only two would be minimally necessary to solve the problem. But deferring to a templated implementation is an acceptable option and has been improved by no longer requiring casts. We believe that the problem is minimal.

§ 4.3 Not quite static, not quite non-static

The status quo is that all member functions are either static member functions or non-static member functions. A member function with an explicit object parameter (see the wording overview for why "object parameter" rather than "this parameter" or something else) is a third kind of member function that's sort of halfway in between those two. They're like semi-static member functions. We'll call them explicit object member functions due to them having an explicit object parameter. You can think of regular non-static member functions as being implicit object member functions.

The high level overview of the design is that from the *outside*, an explicit object member function looks and behaves as much like a non-static member function as possible. You can't take an lvalue to such a member, they have to be invoked like non-static member functions, etc. But from the *inside*, an explicit object member function behaves exactly like a static member function: there is no implicit `this`, your only access to the class object is through the explicit object parameter. The difference is still observable — a pointer to such a function has pointer to function type rather than pointer to member type, but that's about the extent of it.

Some examples of distinctions:

```
struct C {
    void nonstatic_fun();

    void explicit_fun(this C c) {
        auto x = this;      // error: no this
        nonstatic_fun();    // error: no implicit this->, non-static member function needs an object
        c.nonstatic_fun();  // ok

        static_fun(C{});    // ok
        (+static_fun)(C{}); // ok
    }

    static void static_fun(C) {
        explicit_fun();        // error: needs an object
        explicit_fun(C{});     // error: not the right way to provide an object
        auto f = explicit_fun; // error: can't just name such a function, must invoke it
        (+explicit_fun)(C{});  // error: can't convert the name to a pointer

        C{}.explicit_fun();        // ok
        auto p = explicit_fun;     // error: as above
        auto q = &explicit_fun;    // error: can't take a unqualified reference either
        auto r = &C::explicit_fun; // ok
        r(C{});                    // ok
    }

    static void operator()(int);   // error: call operator still can't be static
    void operator()(this C, char); // ok
};

C c;
int (*a)(C) = &C::explicit_fun; // ok
```

```
int (*b)(C) = C::explicit_fun;  // error: can't just name such a function, must invoke it

auto x = c.static_fun;     // ok
auto y = c.explicit_fun;   // error: can't just name such a function, must invoke it
auto z = c.explicit_fun(); // ok
```

The question is: should we describe these new functions as a new kind of *static* member function or as a new kind of *non-static* member function? For the sake of this small section, I'm going to refer to the three kinds of functions as either "legacy static" (status quo static member functions), "legacy non-static" (status quo non-static member functions), or "explicit this" (the ones introduced in this paper).

If we go through several salient aspects of legacy static and legacy non-static member functions, we can see how explicit this functions fare:

	legacy non-static	explicit this	legacy static
Requires/uses an object argument	Yes	Yes	No
Has implicit `this`	Yes	No	No
Can be used to declare operators	Yes	Yes	No
Type of `&C::f`	Pointer-to-Member	Pointer-to-Function	Pointer-to-Function
Does `auto f = x.f;` work?	No	No	Yes
Can the name decay to a function pointer?	No	No	Yes
Can it be virtual	Yes	Maybe	No

If we consider an explicit object member function as being a static member function, as have to answer the question of what to do with the `static` keyword:

```
struct C {
    void f(this C const&);          // proposed ok
    static void g(this C const&);   // what about this?
};
```

If `C::f` is static, then either `C::g` is redundantly static (so the keyword does nothing?) or ill-formed (so the keyword breaks code?). From the user's perspective, `g` behaves like a non-static member function. It needs to be invoked on an object, and it should not matter in most cases whether `g` is implemented with an explicit or implicit object parameter. Annoting such a function as `static` is potentially misleading as it suggests an entirely different usage pattern. We still have to use it as `c.g()` while `C::g(c)` is ill-formed.

Or we go the reverse and say that `C::f` is actually ill-formed and say that you *have* to annotate it as `static` (meaning this feature actually requires two keywords: `static` and `this`). But mandating `static` doesn't work well with lambdas:

```
[](this auto self) static { ... }
```

But we're not especially used to writing these specifiers after the lambda's declarator. `constexpr` is implicit, so is not necessary to write. `consteval` is too new to have much feedback on yet. Mandating `static` here seems actively harmful, while making it optional seems to provide no benefit.

As described later, while explicit object member functions cannot be `virtual` with this proposal, it is possible that they could be in the future. So annotating them as `static` would be exceedingly misleading in this sense.

As a result of all of the above, this paper proposes that:

— a member function declared with an explicit `this` parameter (i.e. an explicit object member function) is a non-static member function,

— an explicit object member function cannot be declared `static` (i.e. it really is a *non-static* member function),

— an explicit object member function cannot be `virtual` (see later section)

— a legacy non-static member function will be called an implicit object member function

Having gone through the wording for this proposal both considering explicit object member functions as static and non-static, there are a lot more rules that apply to both implicit and explicit object member functions (to the exclusion of static member functions) than there are that apply to both C++17 static member functions and explicit object member functions. As such, the amount of wording to be done is also smaller if we consider explicit object member functions as non-static.

4.3.1 Implicit this access

One question came up over the course of implementing this feature which was whether or not there should be implicit this syntax for invoking an explicit object member function from an implicit object member function. That is:

```
struct D {
    void explicit_fun(this D const&);

    void implicit_fun() const {
        this->explicit_fun(); // obviously ok
        explicit_fun();       // but what about this?
    }
};
```

It's tempting to say that given an *explicit* object parameter, we should always require an *explicit* object argument. But one of the major advantages of having the "implicit this" call support in this context is that it allows derived types to not have to know about the implementation choice. As frequently used as a motivating example, we would like `std::optional` to be able to implement its member functions using this language feature if it wants to, without us having to know about it:

```
struct F : std::optional<int>
{
    bool is_big() const {
        // if this language feature required explicit call syntax, then this
        // call would be valid if and only if the particular implementation of
        // optional did _not_ use this feature. This is undesirable
        return value() > 42;
    }
};
```

Or, more generally, the implementation strategy of a particular type's member function should not be relevant for users deriving from that type. So "implicit this" stays.

4.3.2 Explicit object parameter and virtual

In this proposal, member functions with an explicit object parameter cannot be `virtual`. But explicit object member functions are intended to be a substitute for implicit object member functions, so why not?

Many of the motivating use-cases for this feature involve templates, which make the question of whether or not to allow virtual moot. However, several people have expressed an interest in using such syntax as their sole choice for writing any non-static member functions as a style choice. While we are not here to endorse or reject any particular style choice, we should at least consider to what extent this proposal can support them.

To that end, the question of allowing explicit object member functions to be virtual comes up. Consider:

```
struct B {
    virtual void f(); // no ref-qualifier
};

struct D : B {
    void f(this D&) override; // ok?
};
```

There are two potential issues with this approach.

The first is that these really do mean slightly different things. `B().f()` is a valid expression. While `B::f`'s implicit object parameter has type `B&`, it can be invoked with any value category of `B`. But `D().f()` is ill-formed, we have no such value category carve-out for the explicit object parameter.

This particular issue doesn't seem terribly problematic, since the ability to invoke virtual member functions on rvalues of (statically) derived type isn't really the primary (or any kind of) motivation for virtual functions. But it is an issue.

The second, and larger, issue is a question of calling convention. What would the calling convention be? If the calling convention of an explicit object member function is the same as that of a free function, would the virtual dispatch work by generating a forwarding thunk to do calling convention adjustment or would we emit `D::f` twice with two different calling conventions? Or, if the calling convention is the same as that of a member function, do we introduce a generalized pointer-to-member function type to properly account for by-value object parameters? Calling convention mismatch is a problem, and introducing a new kind of pointer-to-member type seems like a fairly large and potentially viral change.

Notably, neither of these problems exist if we only allow overrides of the same *kind* of member function. That is, explicit object member functions can only override other explicit object member functions and implicit object member functions can only override other implicit object member functions:

```
struct B2 {
    virtual void f();
    virtual void g(this B2 const&, int);
};

struct D2 : B2 {
    void f() override;                    // ok
    void f(this D2 const&) override;      // error

    void g(int) const& override;          // error
    void g(this D2 const&, int) override; // ok
};
```

Such a direction would introduce a second observable difference between implicit and explicit object member functions: the type of a pointer to such a function and the override mechanism.

However, such a direction also doesn't provide the user with any ability that they didn't have before. It would purely be a style choice. As such, we don't consider the question of allowing `virtual` to be especially important at this time.

Note that the paper currently allows this:

```
struct B3 {
    virtual void f();
};

struct D3 : B3 {
    void f(this D3&); // ok, does not override B3::f
};
```

So while this keeps the door open to a future extension that would allow explicit object member functions to be `virtual`, it would only allow same-kind overriding going forward (since allowing mixed-kind overriding could potentially change the meaning of code written between now and then).

§ 5 Real-World Examples

What follows are several examples of the kinds of problems that can be solved using this proposal.

§ 5.1 Deduplicating Code

This proposal can de-duplicate and de-quadruplicate a large amount of code. In each case, the single function is only slightly more complex than the initial two or four, which makes for a huge win. What follows are a few examples of ways to reduce repeated code.

This particular implementation of `optional` is Simon's, and can be viewed on GitHub. It includes some functions proposed in [P0798R0], with minor changes to better suit this format:

C++17	Proposed
`` class TextBlock { public: char const& operator[](size_t position) const { // ... return text[position]; } char& operator[](size_t position) { return const_cast<char&>(static_cast<TextBlock const&> (this)[position]); } // ... }; ```	``` class TextBlock { public: template <typename Self> auto& operator[](this Self&& self, size_t position) { // ... return self.text[position]; } // ... }; ```
`` template <typename T> class optional { // ... constexpr T* operator->() { return addressof(this->m_value); } constexpr T const* operator->() const { return addressof(this->m_value); } // ... }; ```	``` template <typename T> class optional { // ... template <typename Self> constexpr auto operator->(this Self&& self) { return addressof(self.m_value); } // ... }; ```
`` template <typename T> class optional { // ... constexpr T& operator*() & { return this->m_value; } constexpr T const& operator*() const& { return this->m_value; } constexpr T&& operator*() && { return move(this->m_value); } constexpr T const&& operator*() const&& { return move(this->m_value); } constexpr T& value() & { ```	``` template <typename T> class optional { // ... template <typename Self> constexpr like_t<Self, T>&& operator*(this Self&& self) { return forward<Self>(self).m_value; } template <typename Self> constexpr like_t<Self, T>&& value(this Self&& self) { if (self.has_value()) { return forward<Self>(self).m_value; } throw bad_optional_access(); } // ... }; ```

```
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr T const& value() const& {
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr T&& value() && {
    if (has_value()) {
      return move(this->m_value);
    }
    throw bad_optional_access();
  }

  constexpr T const&& value() const&& {
    if (has_value()) {
      return move(this->m_value);
    }
    throw bad_optional_access();
  }
  // ...
};
```

```
template <typename T>
class optional {
  // ...
  template <typename F>
  constexpr auto and_then(F&& f) & {
    using result =
      invoke_result_t<F, T&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f), **this)
        : nullopt;
  }

  template <typename F>
  constexpr auto and_then(F&& f) && {
    using result =
      invoke_result_t<F, T&&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f),
                 move(**this))
        : nullopt;
  }

  template <typename F>
  constexpr auto and_then(F&& f) const& {
    using result =
      invoke_result_t<F, T const&>;
```

```
template <typename T>
class optional {
  // ...
  template <typename Self, typename F>
  constexpr auto and_then(this Self&& self,
        F&& f) {
    using val = decltype((
        forward<Self>(self).m_value));
    using result = invoke_result_t<F, val>;

    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return self.has_value()
        ? invoke(forward<F>(f),
                 forward<Self>(self).m_value)
        : nullopt;
  }
  // ...
};
```

```
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f), **this)
        : nullopt;
  }

  template <typename F>
  constexpr auto and_then(F&& f) const&& {
    using result =
      invoke_result_t<F, T const&&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f),
                 move(**this))
        : nullopt;
  }
  // ...
};
```

There are a few more functions in [P0798R0] responsible for this explosion of overloads, so the difference in both code and clarity is dramatic.

For those that dislike returning auto in these cases, it is easy to write a metafunction matching the appropriate qualifiers from a type. It is certainly a better option than blindly copying and pasting code, hoping that the minor changes were made correctly in each case.

§ 5.2 CRTP, without the C, R, or even T

Today, a common design pattern is the Curiously Recurring Template Pattern. This implies passing the derived type as a template parameter to a base class template as a way of achieving static polymorphism. If we wanted to simply outsource implementing postfix incrementation to a base, we could use CRTP for that. But with explicit objects that already deduce to the derived objects, we don't need any curious recurrence — we can use standard inheritance and let deduction do its thing. The base class doesn't even need to be a template:

C++17

```
template <typename Derived>
struct add_postfix_increment {
    Derived operator++(int) {
        auto& self = static_cast<Derived&>
        (*this);

        Derived tmp(self);
        ++self;
        return tmp;
    }
};

struct some_type :
        add_postfix_increment<some_type> {
    some_type& operator++() { ... }
};
```

Proposed

```
struct add_postfix_increment {
    template <typename Self>
    auto operator++(this Self&& self, int) {
        auto tmp = self;
        ++self;
        return tmp;
    }
};


struct some_type : add_postfix_increment {
    some_type& operator++() { ... }
};
```

The proposed examples aren't much shorter, but they are certainly simpler by comparison.

5.2.1 Builder pattern

Once we start to do any more with CRTP, complexity quickly increases, whereas with this proposal, it stays remarkably low.

Let's say we have a builder that does multiple things. We might start with:

```
struct Builder {
  Builder& a() { /* ... */; return *this; }
  Builder& b() { /* ... */; return *this; }
  Builder& c() { /* ... */; return *this; }
};

Builder().a().b().a().b().c();
```

But now we want to create a specialized builder with new operations `d()` and `e()`. This specialized builder needs new member functions, and we don't want to burden existing users with them. We also want `Special().a().d()` to work, so we need to use CRTP to *conditionally* return either a `Builder&` or a `Special&`:

C++17

```
template <typename D=void>
class Builder {
  using Derived = conditional_t<is_void_v<D>,
        Builder, D>;
  Derived& self() {
    return *static_cast<Derived*>(this);
  }

public:
  Derived& a() { /* ... */; return self(); }
  Derived& b() { /* ... */; return self(); }
  Derived& c() { /* ... */; return self(); }
};

struct Special : Builder<Special> {
  Special& d() { /* ... */; return *this; }
  Special& e() { /* ... */; return *this; }
};

Builder().a().b().a().b().c();
Special().a().d().e().a();
```

Proposed

```
struct Builder {
    template <typename Self>
    Self& a(this Self&& self) { /* ... */;
        return self; }

    template <typename Self>
    Self& b(this Self&& self) { /* ... */;
        return self; }

    template <typename Self>
    Self& c(this Self&& self) { /* ... */;
        return self; }
};

struct Special : Builder {
    Special& d() { /* ... */; return *this; }
    Special& e() { /* ... */; return *this; }
};

Builder().a().b().a().b().c();
Special().a().d().e().a();
```

The code on the right is dramatically easier to understand and therefore more accessible to more programmers than the code on the left.

But wait! There's more!

What if we added a *super*-specialized builder, a more special form of `Special`? Now we need `Special` to opt-in to CRTP so that it knows which type to pass to `Builder`, ensuring that everything in the hierarchy returns the correct type. It's about this point that most programmers would give up. But with this proposal, there's no problem!

C++17

```
template <typename D=void>
class Builder {
protected:
```

Proposed

```
struct Builder {
    template <typename Self>
    Self& a(this Self&& self) { /* ...
        */; return self; }
```

```
  using Derived = conditional_t<is_void_v<D>,
        Builder, D>;
  Derived& self() {
    return *static_cast<Derived*>(this);
  }

public:
  Derived& a() { /* ... */; return self(); }
  Derived& b() { /* ... */; return self(); }
  Derived& c() { /* ... */; return self(); }
};

template <typename D=void>
struct Special
  : Builder<conditional_t<is_void_v<D>,Special<D>,D>
{
  using Derived = typename
        Special::Builder::Derived;
  Derived& d() { /* ... */; return this->self(); }
  Derived& e() { /* ... */; return this->self(); }
};

struct Super : Special<Super>
{
    Super& f() { /* ... */; return *this; }
};

Builder().a().b().a().b().c();
Special().a().d().e().a();
Super().a().d().f().e();
```

```
    template <typename Self>
    Self& b(this Self&& self) { /* ...
        */; return self; }

    template <typename Self>
    Self& c(this Self&& self) { /* ...
        */; return self; }
};

struct Special : Builder {
    template <typename Self>
    Self& d(this Self&& self) { /* ...
        */; return self; }

    template <typename Self>
    Self& e(this Self&& self) { /* ...
        */; return self; }
};

struct Super : Special {
    template <typename Self>
    Self& f(this Self&& self) { /* ...
        */; return self; }
};

Builder().a().b().a().b().c();
Special().a().d().e().a();
Super().a().d().f().e();
```

The code on the right is much easier in all contexts. There are so many situations where this idiom, if available, would give programmers a better solution for problems that they cannot easily solve today.

Note that the `Super` implementations with this proposal opt-in to further derivation, since it's a no-brainer at this point.

5.3 Recursive Lambdas

The explicit object parameter syntax offers an alternative solution to implementing a recursive lambda as compared to [P0839R0], since now we've opened up the possibility of allowing a lambda to reference itself. To do this, we need a way to *name* the lambda.

```
// as proposed in P0839
auto fib = [] self (int n) {
    if (n < 2) return n;
    return self(n-1) + self(n-2);
};

// this proposal
auto fib = [](this auto self, int n) {
    if (n < 2) return n;
    return self(n-1) + self(n-2);
};
```

This works by following the established rules. The call operator of the closure object can also have an explicit object parameter, so in this example, `self` is the closure object.

In San Diego, issues of implementability were raised. The proposal ends up being implementable. See the lambda FAQ entry for details.

Combine this with the new style of mixins allowing us to automatically deduce the most derived object, and you get the following example — a simple recursive lambda that counts the number of leaves in a tree.

```
struct Leaf { };
struct Node;
using Tree = variant<Leaf, Node*>;
struct Node {
    Tree left;
    Tree right;
};

int num_leaves(Tree const& tree) {
    return visit(overload(        // <-----------------------------------+
        [](Leaf const&) { return 1; },                                   //      |
        [](this auto const& self, Node* n) -> int {                      //      |
            return visit(self, n->left) + visit(self, n->right); // <----+
        }
    ), tree);
}
```

In the calls to `visit`, `self` isn't the lambda; `self` is the `overload` wrapper. This works straight out of the box.

§ 5.4 By-value member functions

This section presents some of the cases for by-value member functions.

§ 5.4.1 For move-into-parameter chaining

Say you wanted to provide a `.sorted()` method on a data structure. Such a method naturally wants to operate on a copy. Taking the parameter by value will cleanly and correctly move into the parameter if the original object is an rvalue without requiring templates.

```
struct my_vector : vector<int> {
  auto sorted(this my_vector self) -> my_vector {
    sort(self.begin(), self.end());
    return self;
  }
};
```

§ 5.4.2 For performance

It's been established that if you want the best performance, you should pass small types by value to avoid an indirection penalty. One such small type is `std::string_view`. Abseil Tip #1 for instance, states:

> Unlike other string types, you should pass `string_view` by value just like you would an `int` or a `double` because `string_view` is a small value.

There is, however, one place today where you simply *cannot* pass types like `string_view` by value: to their own member functions. The implicit object parameter is always a reference, so any such member functions that do not get inlined incur a double indirection.

As an easy performance optimization, any member function of small types that does not perform any modifications can take the object parameter by value. Here is an example of some member functions of `basic_string_view` assuming that we are just using `charT const*` as `iterator`:

```
template <class charT, class traits = char_traits<charT>>
class basic_string_view {
private:
    const_pointer data_;
    size_type size_;
```

```
public:
    constexpr const_iterator begin(this basic_string_view self) {
        return self.data_;
    }

    constexpr const_iterator end(this basic_string_view self) {
        return self.data_ + self.size_;
    }

    constexpr size_t size(this basic_string_view self) {
        return self.size_;
    }

    constexpr const_reference operator[](this basic_string_view self, size_type pos) {
        return self.data_[pos];
    }
};
```

Most of the member functions can be rewritten this way for a free performance boost.

The same can be said for types that aren't only cheap to copy, but have no state at all. Compare these two implementations of `less_than`:

C++17	Proposed
`struct less_than {` `  template <typename T, typename U>` `  bool operator()(T const& lhs, U const& rhs)` `  {` `  return lhs < rhs;` `  }` `};`	`struct less_than {` `  template <typename T, typename U>` `  bool operator()(this less_than,` `  T const& lhs, U const& rhs) {` `  return lhs < rhs;` `  }` `};`

In C++17, invoking `less_than()(x, y)` still requires an implicit reference to the `less_than` object — completely unnecessary work when copying it is free. The compiler knows it doesn't have to do anything. We *want* to pass `less_than` by value here. Indeed, this specific situation is the main motivation for [P1169R0].

5.4.3 For lifetime management of coroutines

One issue with coroutines is dealing with dangling references. This issue isn't specific to coroutines at all, it's simply that coroutines provide another venue for dangling references that takes some adjusting to.

One way to avoid dealing with dangling references is to, quite simply, not have references:

Dangles	Doesn't dangle
`auto always(int const& val) ->` `  std::generator<int> {` `  for (;;) {` `  co_yield val;` `  }` `}` `// 'val' above ends up being a dangling` `  reference` `for (int i : always(42)) { ... }`	`auto always(int val) -> std::generator<int> {` `  for (;;) {` `  co_yield val;` `  }` `}` `// ok: val is copied` `for (int i : always(42)) { ... }`

This general approach works for every function parameter — except the implicit object parameter, which is always a reference. But this proposal allows you to to avoid this dangling even for member function coroutines by also taking the

object parameter by value:

Dangles	Doesn't dangle
`struct C {` `  int val;` `  auto always() const -> std::generator<int>` `  {` `  for (;;) {` `  co_yield val;` `  }` `  }` `};` `// 'this' above ends up being a dangling` `  pointer` `for (int i : C{42}.always()) { ... }`	`struct C {` `  int val;` `  auto always(this C c) ->` `  std::generator<int> {` `  for (;;) {` `  co_yield c.val;` `  }` `  }` `};` `// ok: C is copied` `for (int i : C{42}.always()) { ... }`

§ 5.5 SFINAE-friendly callables

A seemingly unrelated problem to the question of code quadruplication is that of writing numerous overloads for function wrappers, as demonstrated in [P0826R0]. Consider what happens if we implement `std::not_fn()` as currently specified:

```
template <typename F>
class call_wrapper {
    F f;
public:
    // ...
    template <typename... Args>
    auto operator()(Args&&... ) &
        -> decltype(!declval<invoke_result_t<F&, Args...>>());

    template <typename... Args>
    auto operator()(Args&&... ) const&
        -> decltype(!declval<invoke_result_t<F const&, Args...>>());

    // ... same for && and const && ...
};

template <typename F>
auto not_fn(F&& f) {
    return call_wrapper<decay_t<F>>{forward<F>(f)};
}
```

As described in the paper, this implementation has two pathological cases: one in which the callable is SFINAE-unfriendly, causing the call to be ill-formed where it would otherwise work; and one in which overload is deleted, causing the call to fall back to a different overload when it should fail instead:

```
struct unfriendly {
    template <typename T>
    auto operator()(T v) {
        static_assert(is_same_v<T, int>);
        return v;
    }

    template <typename T>
    auto operator()(T v) const {
        static_assert(is_same_v<T, double>);
        return v;
    }
```

```
};

struct fun {
    template <typename... Args>
    void operator()(Args&&...) = delete;

    template <typename... Args>
    bool operator()(Args&&...) const { return true; }
};

std::not_fn(unfriendly{})(1); // static assert!
                              // even though the non-const overload is viable and would be the
                              // best match, during overload resolution, both overloads of
                              // unfriendly have to be instantiated - and the second one is a
                              // hard compile error.

std::not_fn(fun{})();         // ok!? Returns false
                              // even though we want the non-const overload to be deleted, the
                              // const overload of the call_wrapper ends up being viable - and
                              // the only viable candidate.
```

Gracefully handling SFINAE-unfriendly callables is **not solvable** in C++ today. Preventing fallback can be solved by the addition of another four overloads, so that each of the four *cv*/ref-qualifiers leads to a pair of overloads: one enabled and one `deleted`.

This proposal solves both problems by allowing `this` to be deduced. The following is a complete implementation of `std::not_fn`. For simplicity, it makes use of `BOOST_HOF_RETURNS` from Boost.HOF to avoid duplicating expressions:

```
template <typename F>
struct call_wrapper {
  F f;

  template <typename Self, typename... Args>
  auto operator()(this Self&& self, Args&&... args)
    BOOST_HOF_RETURNS(
      !invoke(
        forward<Self>(self).f,
        forward<Args>(args)...))
};

template <typename F>
auto not_fn(F&& f) {
  return call_wrapper<decay_t<F>>{forward<F>(f)};
}
```

Which leads to:

```
not_fn(unfriendly{})(1); // ok
not_fn(fun{})();         // error
```

Here, there is only one overload with everything deduced together. The first example now works correctly. `Self` gets deduced as `call_wrapper<unfriendly>`, and the one `operator()` will only consider `unfriendly`'s non-`const` call operator. The `const` one is never even considered, so it does not have an opportunity to cause problems.

The second example now also fails correctly. Previously, we had four candidates. The two non-`const` options were removed from the overload set due to `fun`'s non-`const` call operator being `deleted`, and the two `const` ones which were viable. But now, we only have one candidate. `Self` is deduced as `call_wrapper<fun>`, which requires `fun`'s non-`const` call operator to be well-formed. Since it is not, the call results in an error. There is no opportunity for fallback since only one overload is ever considered.

This singular overload has precisely the desired behavior: working for `unfriendly`, and not working for `fun`.

This could also be implemented as a lambda completely within the body of `not_fn`:

```
template <typename F>
auto not_fn(F&& f) {
    return [f=forward<F>(f)](this auto&& self, auto&&.. args)
        BOOST_HOF_RETURNS(
            !invoke(
                forward_like<decltype(self)>(f),
                forward<decltype(args)>(args)...))
        ;
}
```

6 Frequently Asked Questions

6.1 On the implementability of recursive lambdas

In San Diego, 2018, there was a question of whether recursive lambdas are implementable. They are, details follow.

The specific issue is the way lambdas are parsed. When parsing a *non-generic* lambda function body with a default capture, the type of `this_lambda` would not be dependent, because the body is *not a template*. This leads to `sizeof(this_lambda)` not being dependent either, and must therefore have an answer - and yet, it cannot, as the lambda capture is not complete, and therefore, the type of `this_lambda` is not complete.

This is a huge issue for any proposal of recursive lambdas that includes non-generic lambdas.

Notice, however, that the syntax this paper proposes is the following:

```
auto fib = [](this auto&& self, int n) {
  if (n < 2) return n;
  return self(n-1) + self(n-2);
}
```

There is, quite obviously, no way to spell a non-generic lambda, because the lambda type is unutterable. `self`'s type is always dependent.

This makes expressions depending on `self` to be parsed using the regular rules of the language. Expressions involving `self` become dependent, and the existing language rules apply, which means both nothing new to implement, and nothing new to teach.

This proposal is therefore implementable, unlike any other we've seen to date. We would really like to thank Daveed Vandevoorde for thinking through this one with us in Aspen 2019.

6.2 Would library implementers use this

In Kona, EWGI asked us to see whether library implementors would use this. The answer seems to be a resounding yes.

We have heard from Casey Carter and Jonathan Wakely that they are interested in this feature. Also, on the ewg/lewg mailing lists, this paper comes up as a solution to a surprising number of questions, and gets referenced in many papers-in-flight. A sampling of papers:

— [P0798R3]

— [P1221R1]

In Herb Sutter's "Name 5 most important papers for C++", 10 out of 289 respondents chose it. Given that the cutoff was 5, and that modules, throwing values, contracts, reflection, coroutines, linear algebra, and pattern matching were all in that list, I find the result a strong indication that it is wanted.

We can also report that Gašper is dearly missing this feature in libciabatta, a mixin support library, as well as his regular work writing libraries.

On the question of whether this would get used in the standard library interfaces, the answer was “not without the ability to constrain the deduced type”, which is a feature C++ needs even without this paper, and is an orthogonal feature. The same authors were generally very enthusiastic about using this feature in their implementations.

§ 6.3 Function Pointer Types

A valid question to ask is what should be the type of this-annotated functions that have a member function equivalent? There are only two options, each with a trade-off. Please assume the existence of these three functions:

```
struct Y {
    int f(int, int) const&;         // exists
    int g(this Y const&, int, int); // this paper
    int h(this Y, int, int);        // this paper, by value
};
```

`g` has a current equivalent (`f`), while `h` does not. `&Y::h`'s type *must* be a regular function pointer.

If we allow `g`'s type to be a pointer-to-member-function, we get non-uniformity between the types of `h` and `g`. We also get implementation issues because the types a template can result in are non-uniform (is this a template for a member function or a free function? Surprise, it's both!).

We also get forward compatibility with any concievable proposal for extension methods - those will *also* have to be free functions by necessity, for roughly the same reasons.

The paper originally proposed it the other way, but this was changed to the current wording through EWG input in Cologne, 2018.

§ 6.4 Deducing to Base-Class Pointer

One of the pitfalls of having a deduced object parameter is when the intent is solely to deduce the *cv*-qualifiers and value category of the object parameter, but a derived type is deduced as well — any access through an object that might have a derived type could inadvertently refer to a shadowed member in the derived class. While this is desirable and very powerful in the case of mixins, it is not always desirable in other situations. Superfluous template instantiations are also unwelcome side effects.

One family of possible solutions could be summarized as **make it easy to get the base class pointer**. However, all of these solutions still require extra instantiations. For `optional::value()`, we really only want four instantiations: `&`, `const&`, `&&`, and `const&&`. If something inherits from `optional`, we don't want additional instantiations of those functions for the derived types, which won't do anything new, anyway. This is code bloat.

This is already a problem for free-function templates: The authors have heard many a complaint about it from library vendors, even before this paper was introduced, as it is desirable to only deduce the ref-qualifier in many contexts. Therefore, it might make sense to tackle this issue in a more general way. A complementary feature could be proposed to constrain *type deduction*.

The authors strongly believe this feature is orthogonal. However, hoping that mentioning that solutions are in the pipeline helps gain consensus for this paper, we mention one solution here. The proposal is in early stages, and is not in the pre-belfast mailing. It will be present in the post-belfast mailing: computed deduction

§ 6.5 Was this syntax considered?

In the course of working on this paper, many different syntaxes were considered for how to properly express this idea. Those various options have been culled from previous revisions of the paper, but we should have always kept them in for posterity. So we're adding them in here.

[P0847R0] originally proposed the syntax as:

```
struct X {
    // an explicit object parameter named self
    void f(X& this self);

    // an unnamed, deduced explicit object parameter
    template <typename Self>
    void g(Self&& this);
};
```

[P0847R1] considered four potential syntaxes. We'll present them here again with a trivial example that simply returns a member variable, without any deduction.

1. An explicit `this`-annotated parameter. We took the R0 syntax and reordered it so that `this` precedes the parameter declaration rather than being in the middle of it.
2. An explicit parameter *named* `this`. That is, the first parameter *must* be named `this`, which would then become an object rather than a pointer.
3. Having a trailing type (rather than just cv- and ref-qualifiers) with an identifier
4. Having a trailing type (as above) just without an identifier.

Putting them all in a single example:

```
struct A {
    // #1
    int get(this A const& a) { return a.i; }

    // #2
    int get(A const& this) { return this.i; }

    // #3
    int get() A const& self { return self.i; }

    // #4
    int get() A const& { return this->i; }

    int i;
};
```

We ended up settling on option `#1` (as can be seen from the rest of this paper). The syntax isn't especially adventurous and solves all the use-cases we want to solve, and compares very favorably to the rest.

While options `#3` and `#4` keep the cv- and ref-qualifiers where they are today, so in some sense they are more familiar, there were other issues with them overall that led us to where we are today. `#3` has parsing issues, `#4` doesn't work for lambdas and would have to have `this` be potentially dependent, neither allows by-value member functions. `#2` would provide meaning to parameter names that doesn't currently exist today, would give a very different interpretation to `this` than the one we've always had in C++, and would have fairly poor interaction with lambdas that may need to capture `this`.

§ 7 Reflection

One question that has come up periodically is: would we still need this language feature if we had a reflection facility that offered code injection (as described in [P2237R0])? We can answer this question by going through the use-cases we've presented in this paper and try to figure out how well they could be resolved by a code-injection facility.

§ 7.1 Deduplicating Code

Of the five use-cases, this one is the most up in the air. This one seems unlikely to be well-handled by code injection, but it really depends on the kinds of facilities injection will end up allowing. Let's consider the simplest possible case:

```
template <typename T>
struct not_very_optional {
    T value;

    auto get() & -> T& { return value; }
    auto get() const& -> T const& { return value; }
    auto get() && -> T const& { return std::move(value); }
    auto get() const&& -> T const&& { return std::move(value); }
};
```

As presented earlier, one way to do this is to implement three of these in terms of the fourth. For this case, this is something that potentially could be handled through injection, as in this way demonstrated on the reflectors by Ville Voutilainen:

```
template <typename T>
struct not_very_optional {
    T value;

    auto get() & -> T& { return value; }

    consteval {
        std::meta::gen_crval_overloads(reflexpr(not_very_optional::get));
    }
};
```

Although it's not clear if this pattern would work for more complex overload sets. As in, if the different overloads needed different constraints or had different `noexcept` specifications:

```
template <typename T>
struct still_not_very_optional {
    auto map(invocable<T&> auto&&) &;
    auto map(invocable<T const&> auto&&) const&;
    auto map(invocable<T&&> auto&&) &&;
    auto map(invocable<T const&&> auto&&) const&&;
};
```

This doesn't really translate in the `gen_crval_overloads` model. You could do this sort of thing with macros:

```
#define INJECT_QUALS(X) X(&) X(const&) X(&&) X(const&&)

template <typename T>
struct still_not_very_optional {
    #define MAP_QUALS(q) auto map(invocable<T q> auto&&) q;
    INJECT_QUALS(MAP_QUALS)
};
```

Which suggests a potential code injection direction if we could inject qualifiers somehow, which is a feature that the Metaprogramming paper does not mention, and it is unclear if that is a direction that will be pursued.

As a result, we have to state that reflection as proposed thus far would not really address this use-case.

§ 7.2 Better mixin support

Deducing this provides us a better way to write mixins. But mixins are an especially clear use-case for code injection, and one that code injection could easily provide a superior alternative.

We presented an example with postfix increment earlier in the paper, here is how that example could look with code injection:

Proposed in this paper	With reflection
`struct add_postfix_increment {` `  template <typename Self>` `  auto operator++(this Self&& self, int) {` `  auto tmp = self;` `  ++self;` `  return tmp;` `  }` `};` `struct some_type : add_postfix_increment {` `  some_type& operator++() { ... }` `};`	`constexpr auto add_postfix_increment =` `  <struct T{` `  T operator++(int) {` `  T tmp = *this;` `  ++*this;` `  return tmp;` `  }` `  }>;` `struct some_type {` `  some_type& operator++() { ... }` `  << add_postfix_increment;` `};`

Assuming this is roughly how the code injection facility will look, we expect the code on the right to be preferred in many use-cases. While obviously novel for C++, it's also simpler (there are no templates) and it is a more direct and less intrusive way to add functionality to a class (`some_type` no longer needs to have a base class, which is a meaningful benefit).

7.3 The other three

The other three use-cases presented in this paper are recursive lambdas, by-value member functions, and the ability to properly create SFINAE-friendly call wrappers. What all of these use-cases have in common is that they are all cases you cannot write today. You cannot write a recursive lambda because you have no way of naming the lambda itself from its body, you cannot write a by-value member function since the object parameter of non-static member functions is always a reference, and you cannot create SFINAE-friendly call wrappers since you cannot write the wrapper as a single function template.

The ability to deduce this — to treat the object parameter as a first-class function parameter — is a new language feature that allows us to do all of these things. It gives us the ability to name the lambda, to take the object parameter by value, and to write a single function template for call wrappers rather than writing four different call operators.

Code injection facilities can only inject code that you could already write yourself by hand. As such, no matter where reflection takes us, it could not provide solutions for these problems since they fundamentally require new language support.

Potentially, reflection could provide some magic `std::meta::get_current_lambda()` function that when invoked from within a lambda body could give you access to the lambda itself. But this would have to be a facility provided by a compiler intrinsic and seems like an especially unsatisfying solution as compared to the one presented in this paper.

7.4 Reflection vs deducing this

Of the five use-cases presented in this paper, we expect Reflection to provide a superior solution to one of them. But it basically cannot solve three of them, and it is unclear to what extent it would be able to provide a satisfactory solution to the fifth. As a result, Reflection can't really be a substitute for this proposal on the whole, even if we could get the facilities described in the Metaprogramming paper right away.

8 Implementation

This has been implemented in the EDG front end, with gracious help and encouragement from Daveed Vandevoorde. Implementation didn't turn up any notable issues.

§ 9 Proposed Wording

§ 9.1 Overview

The status quo here is that a member function has an *implicit object parameter*, always of reference type, which the *implied object argument* is bound to. The obvious name for what this paper is proposing is, then, the *explicit object parameter*. The problem with these names is: well, what is an object parameter? A parameter that takes an object? Isn't that most parameters?

However, calling it a *this parameter* is confusing in a different way: `this` is a pointer, and the parameter in question would always be either a reference type (as is always the case today) or a value (as is possible with this feature), never a pointer.

We considered a lot of other terms - self parameter, selector parameter, instance parameter, subject parameter, target parameter. But we kind of feel like maybe "object parameter" is actually fine? In classical OO, perhaps "subject" might be better than "object", but maybe object is good enough.

§ 9.2 Wording

[Editor's note: This paper introduces many new terms that are defined in [dcl.dcl] - so even though the wording here is presented in standard layout order (we obviously want to ensure that `is_standard_layout<P0847>` is `true`), it may be helpful to refer to those definitions when reviewing the wording.

One decision was to introduce the term *object parameter* as the union of explicit object parameter (new in this paper) and implicit object parameter (preexisting). This is the difference between making changes like "implicit or explicit object parameter" in a bunch of places vs "~~implicit~~ object parameter" in a bunch of places.]

§ 9.2.1 Wording in 6 [basic]

Extend the definition of correspond in 6.4.1 [basic.scope.scope]/3 to check the implicit/explicit object parameter types:

a Two non-static member functions have *corresponding object parameters* if:

(a.1) — exactly one is an implicit object member function with no *ref-qualifier* and the types of their object parameters, are removing top-level references, are the same, or

(a.2) — their object parameters have the same type.

b Two non-static member function templates have *corresponding object parameters* if:

(b.1) — exactly one is an implicit object member function with no *ref-qualifier* and the types of their object parameters, after removing any references, are equivalent, or

(b.2) — the types of their object parameters are equivalent.

3 Two declarations *correspond* if they (re)introduce the same name, both declare constructors, or both declare destructors, unless

(3.1) — either is a *using-declarator*, or

(3.2) — one declares a type (not a *typedef-name*) and the other declares a variable, non-static data member other than of an anonymous union ([class.union.anon]), enumerator, function, or function template, or

(3.3) — each declares a function or function template, except when

(3.3.1) — both declare functions with the same non-object-parameter-type-list[21], equivalent ([temp.over.link]) trailing *requires-clause*s (if any, except as specified in [temp.friend]), and, if both are non-static

members, ~~the same *cv-qualifiers* (if any) and *ref-qualifier* (if both have one)~~ they have corresponding object parameters, or

(3.3.2) — both declare function templates with equivalent non-object-parameter-type-lists, return types (if any), *template-head*s, and trailing *requires-clause*s (if any), and, if both are non-static members, ~~the same *cv-qualifiers* (if any) and *ref-qualifier* (if both have one)~~ they have corresponding object parameters.

and extend the example:

```
  typedef int Int;
  enum E : int { a };
  void f(int);            // #1
  void f(Int) {}          // defines #1
  void f(E) {}            // OK: another overload

  struct X {
    static void f();
    void f() const;       // error: redeclaration
    void g();
    void g() const;       // OK
    void g() &;           // error: redeclaration

+   void h(this X&, int);
+   void h(int) &&;               // OK: another overload
+   void j(this const X&);
+   void j() const&;              // error: redeclaration
+   void k();
+   void k(this X&);              // error: redeclaration
  };
```

In the C++17 wording, we had this example [over.load]/2.3 in N4659:

```
class Y {
  void h() &;
  void h() const &;             // OK
  void h() &&;                  // OK, all declarations have a ref-qualifier
  void i() &;
  void i() const;               // ill-formed, prior declaration of i
                                // has a ref-qualifier
};
```

The `Y::i` example actually becomes well-formed after [P1787R6] (see Davis' email to core), since the prior wording did not consider *cv*-qualifiers but the new wording does. The wording change above preserves Davis' change, the above is allowed.

In regards to examples like:

```
struct A {
    void f();
    void f() &;
};

struct B {
    void f();
    void f() &&;
};
```

There isn't much reason that the absence of a *ref-qualifier* is more `&`-ish than `&&`-ish. So both of these pairs of function declarations should be rejected. It would be nice to *just* talk about the type of the object parameter, but this would only reject `A` but not `B`.

Hence the whole paragraph about corresponding object parameters. If exactly one is a legacy non-static member function with no *ref-qualifier*, we have to ignore the reference part of the type. That would end up with both of these corresponding.

For an example like this:

```
struct C {
    void f();
    void f(this C);
};
```

This is technically differentiable by overload resolution:

```
struct D : C { };

const C cc;
cc.f(); // calls second overload
D d;
d.f();  // calls first overload
```

But doesn't seem especially meaningful to support either, so should be rejected by the above rule.

9.2.2 Wording in 7 [expr]

Change 7.5.2 [expr.prim.this]/1 and /3:

1 The keyword `this` names a pointer to the object for which ~~a non-static~~ an implicit object member function ([class.mfct.non-static]) is invoked or a non-static data member's initializer ([class.mem]) is evaluated.

3 It shall not appear within the declaration of either a static member function or an explicit object member function of the current class (although its type and value category are defined within ~~a static~~ such member functions as they are within ~~a non-static~~ an implicit object member function).

Change 7.5.4 [expr.prim.id]/2 to properly account for lambdas with an explicit object parameter:

> The result is the entity denoted by the identifier. If the entity is a local entity and naming it from outside of an unevaluated operand within the declarative region where the *unqualified-id* appears would result in some intervening *lambda-expression* capturing it by copy ([expr.prim.lambda.capture]), the type of the expression is the type of a class member access expression ([expr.ref]) naming the non-static data member that would be declared for such a capture in the ~~closure object~~ object parameter ([dcl.fct]) of the function call operator of the innermost such intervening *lambda-expression*.

Change 7.5.5 [expr.prim.lambda]/3:

3 In the *decl-specifier-seq* of the *lambda-declarator*, each *decl-specifier* shall be one of `mutable`, `constexpr`, or `consteval`. If the *lambda-declarator* contains an explicit object parameter ([dcl.fct]), then no *decl-specifier* in the *decl-specifier-seq* shall be `mutable`. [Note: The trailing requires-clause is described in [dcl.decl]. — end note]

Extend the example in 7.5.5.2 [expr.prim.lambda.closure]/3:

```
auto glambda = [](auto a, auto&& b) { return a < b; };
bool b = glambda(3, 3.14);                                  // OK

auto vglambda = [](auto printer) {
  return [=](auto&& ... ts) {                               // OK: ts is a function
      parameter pack
    printer(std::forward<decltype(ts)>(ts)...);

    return [=]() {
      printer(ts ...);
```

```
      };
    };
  };
  auto p = vglambda( [](auto v1, auto v2, auto v3)
                     { std::cout << v1 << v2 << v3; } );
  auto q = p(1, 'a', 3.14);                                   // OK: outputs 1a3.14
  q();                                                        // OK: outputs 1a3.14
+
+ auto fact = [](this auto self, int n) -> int {              // OK: explicit object
        parameter
+    return (n <= 1) ? 1 : n * self(n-1);
+ };
+ std::cout << fact(5);                                       // OK: outputs 120
```

Add a new paragraph after 7.5.5.2 [expr.prim.lambda.closure]/3:

3* Given a lambda with a *lambda-capture*, the type of the explicit object parameter, if any, of the lambda's function call operator (possibly instantiated from a function call operator template) shall be either:

(3*.1) — the closure type,

(3*.2) — a class type derived from the closure type, or

(3*.3) — a reference to a possibly cv-qualified such type.

[*Example*:

```
struct C {
    template <typename T>
    C(T);
};

void func(int i) {
    int x = [=](this auto&&) { return i; }(); // ok
    int y = [=](this C) { return i; }();      // ill-formed
    int z = [](this C) { return 42; }();      // ok
}
```

— *end example*]

Change 7.5.5.2 [expr.prim.lambda.closure]/4:

4 The function call operator or operator template is declared `const` ([class.mfct.non-static]) if and only if the *lambda-expression*'s *parameter-declaration-clause* is not followed by `mutable` and the *lambda-declarator* does not contain an explicit object parameter.

Change 7.6.1.3 [expr.call]/7 to adjust the call arguments by the implied object argument:

7 When a function is called, each parameter ([dcl.fct]) is initialized ([dcl.init], [class.copy.ctor]) with its corresponding argument. If the function is an explicit object member function and there is an implied object argument ([over.call.func]), the list of provided arguments is preceded by the implied object argument for the purposes of this correspondence. If there is no corresponding argument, the default argument for the parameter is used. […] If the function is ~~a non-static~~ an implicit object member function, the `this` parameter of the function is initialized with a pointer to the object of the call, converted as if by an explicit type conversion. [*Note*: There is no access or ambiguity checking on this conversion; the access checking and disambiguation are done as part of the (possibly implicit) class member access operator. See [class.member.lookup], [class.access.base], and [expr.ref]. — end note] When a function is called, the type of any parameter shall not be a class type that is either incomplete or abstract.

Change 7.6.2.2 [expr.unary.op]/3, requiring that taking a pointer to an explicit this function use a *qualified-id*:

3 The result of the unary `&` operator is a pointer to its operand.

(3.1) — If the operand is a *qualified-id* naming a non-static or variant member `m` of some class `C` with type `T`, the result has type "pointer to member of class `C` of type `T`" and is a prvalue designating `C::m`.

(3.2) — Otherwise, if the operand is an lvalue of type `T`, the resulting expression is a prvalue of type "pointer to `T`" whose result is a pointer to the designated object ([intro.memory]) or function. If the operand names an explicit object member function (dcl.fct), the operand shall be a *qualified-id*. [Note 2: In particular, taking the address of a variable of type "cv `T`" yields a pointer of type "pointer to cv `T`". — end note]

(3.3) — Otherwise, the program is ill-formed.

§ 9.2.3 Wording in 9 [dcl.dcl]

In 9.3.4.6 [dcl.fct]/3, allow for a *parameter-declaration* to contain an optional `this` keyword:

parameter-declaration-list:
parameter-declaration
parameter-declaration-list , *parameter-declaration*

parameter-declaration:
attribute-specifier-seq$_{opt}$ `this`$_{opt}$ *decl-specifier-seq declarator*
attribute-specifier-seq$_{opt}$ `this`$_{opt}$ *decl-specifier-seq declarator* `=` *initializer-clause*
attribute-specifier-seq$_{opt}$ `this`$_{opt}$ *decl-specifier-seq abstract-declarator*$_{opt}$
attribute-specifier-seq$_{opt}$ `this`$_{opt}$ *decl-specifier-seq abstract-declarator*$_{opt}$ `=` *initializer-clause*

After 9.3.4.6 [dcl.fct]/5, insert a paragraph describing where a function declaration with an explicit this parameter may appear, and renumber section.

5a An *explicit-object-parameter-declaration* is a *parameter-declaration* with a `this` specifier. An *explicit-object-parameter-declaration* shall appear only as the first *parameter-declaration* of a *parameter-declaration-list* of either:

(5a.1) — a *member-declarator* that declares a member function ([class.mem]), or

(5a.2) — a *lambda-declarator* ([expr.prim.lambda]).

5b A *member-declarator* with an *explicit-object-parameter-declaration* shall not include a *ref-qualifier* or a *cv-qualifier-seq* and shall not be declared `static` or `virtual`.

[*Example*:

```
struct C {
    void f(this C& self);
    template <typename Self>
    void g(this Self&& self, int);

    void h(this C) const; // error: const not allowed here
};

void test(C c) {
    c.f();              // ok: calls C::f
    c.g(42);            // ok: calls C::g<C&>
    std::move(c).g(42); // ok: calls C::g<C>
}
```

— *end example*]

5c A function parameter declared with an *explicit-object-parameter-declaration* is an *explicit object parameter*. An explicit object parameter shall not be a function parameter pack ([temp.variadic]). An *explicit object*

member function is a non-static member function with an explicit object parameter. An *implicit object member function* is non-static member function without an explicit object parameter.

5d The *object parameter* of a non-static member function is either the explicit object parameter or the implicit object parameter ([over.match.funcs]).

5e An *non-object parameter* is a function parameter that is not the explicit object parameter. The *non-object-parameter-type-list* of a member function is the parameter-type-list of that function with the explicit object parameter, if any, omitted. [*Note*: The non-object-parameter-type-list consists of the adjusted types of all the non-object parameters. *-end note*]

Change 9.5.4 [dcl.fct.def.coroutine]/3-4:

3 The *promise type* of a coroutine is `std::coroutine_traits<R, P1, …, Pn>::promise_type`, where `R` is the return type of the function, and `P1…Pn` are the sequence of types of the non-object function parameters, preceded by the type of the ~~implicit~~ object parameter (~~[over.match.funcs]~~ [dcl.fct]) if the coroutine is a non-static member function. The promise type shall be a class type.

4 In the following, `pi` is an lvalue of type `Pi`, where `p1` denotes ~~`*this`~~ the object parameter and `pi+1` denotes the i^{th} non-object function parameter for a non-static member function, and `pi` denotes the i^{th} function parameter otherwise.

§ 9.2.4 Wording in 11 [class]

Change 11.4.3 [class.mfct.non-static]/4-5:

4 [*Note 2*: ~~A non-static~~ An implicit object member function can be declared with *cv*-qualifiers, which affect the type of the `this` pointer ([expr.prim.this]), and/or a *ref-qualifier* ([dcl.fct]); both affect overload resolution ([over.match.funcs]) — *end note*]

5 ~~A non-static~~ An implicit object member function may be declared virtual ([class.virtual]) or pure virtual ([class.abstract]).

Change 11.4.8.3 [class.conv.fct]/1:

1 A member function of a class `X` with a name of the form […] shall have no non-object parameters and specifies a conversion from `X` to the type specified by the *conversion-type-id*, interpreted as a *type-id* ([dcl.name]).

§ 9.2.5 Wording in 12 [over]

Change 12.2.1 [over.match.general]/;

1 Overload resolution is a mechanism for selecting the best function to call given a list of expressions that are to be the arguments of the call and a set of *candidate functions* that can be called based on the context of the call. The selection criteria for the best function are the number of arguments, how well the arguments match the parameter-type-list of the candidate function, how well (for non-static member functions) the object matches the ~~implicit~~ object parameter, and certain other properties of the candidate function.

Change 12.2.2 [over.match.funcs]/2-5:

2 So that argument and parameter lists are comparable within this heterogeneous set, a member function that does not have an explicit object parameter is considered to have an extra first parameter, called the *implicit object parameter*, which represents the object for which the member function has been called. For the purposes of overload resolution, both static and non-static member functions have an ~~implicit~~ object parameter, but constructors do not.

3 Similarly, when appropriate, the context can construct an argument list that contains an *implied object argument* as the first argument in the list to denote the object to be operated on.

4 For ~~non-static~~ implicit object member functions, the type of the implicit object parameter is

(4.1) — "lvalue reference to *cv* `X`" for functions declared without a *ref-qualifier* or with the `&` *ref-qualifier*

(4.2) — "rvalue reference to *cv* `X`" for functions declared with the `&&` *ref-qualifier*

where `X` is the class of which the function is a member and *cv* is the *cv*-qualification on the member function declaration. [Example: For a `const` member function of class `X`, the extra parameter is assumed to have type "lvalue reference to `const X`". — *end example*] For conversion functions that are implicit object member functions, the function is considered to be a member of the class of the implied object argument for the purpose of defining the type of the implicit object parameter. For non-conversion functions that are implicit object member functions introduced by a *using-declaration* into a derived class, the function is considered to be a member of the derived class for the purpose of defining the type of the implicit object parameter. For static member functions, the implicit object parameter is considered to match any object (since if the function is selected, the object is discarded). [*Note*: No actual type is established for the implicit object parameter of a static member function, and no attempt will be made to determine a conversion sequence for that parameter ([over.match.best]). — *end note*]

5 During overload resolution, the implied object argument is indistinguishable from other arguments. The implicit object parameter, however, retains its identity since no user-defined conversions can be applied to achieve a type match with it. For ~~non-static~~ implicit object member functions declared without a *ref-qualifier*, even if the implicit object parameter is not const-qualified, an rvalue can be bound to the parameter as long as in all other respects the argument can be converted to the type of the implicit object parameter. [*Note*: The fact that such an argument is an rvalue does not affect the ranking of implicit conversion sequences. — *end note*]

Add an example to 12.2.2.2.2 [over.call.func]/3:

3 Because of the rules for name lookup, the set of candidate functions consists (1) entirely of non-member functions or (2) entirely of member functions of some class T. In case (1), the argument list is the same as the expression-list in the call. In case (2), the argument list is the *expression-list* in the call augmented by the addition of an implied object argument as in a qualified function call. If the keyword `this` is in scope and refers to class `T`, or a derived class of `T`, then the implied object argument is `(*this)`. If the keyword `this` is not in scope or refers to another class, then a contrived object of type `T` becomes the implied object argument.[113] If the argument list is augmented by a contrived object and overload resolution selects one of the non-static member functions of `T`, the call is ill-formed.

[*Example*:

```
struct C {
    void a();
    void b() {
        a(); // ok, (*this).a()
    }

    void f(this const C&);
    void g() const {
        f();       // ok: (*this).f()
        f(*this);  // error: no viable candidate for (*this).f(*this)
        this->f(); // ok
    }

    static void h() {
        f();       // error: contrived object argument, but overload resolution
                   // picked a non-static member function
        f(C{});    // error: no viable candidate
```

```
        C{}.f();   // ok
    }
};
```

— *end example*]

Add to 12.2.2.2.3 [over.call.object]/3:

3 The argument list submitted to overload resolution consists of the argument expressions present in the function call syntax preceded by the implied object argument `(E)`. [*Note*: When comparing the call against the function call operators, the implied object argument is compared against the ~~implicit~~ object parameter of the function call operator. When comparing the call against a surrogate call function, the implied object argument is compared against the first parameter of the surrogate call function. The conversion function from which the surrogate call function was derived will be used in the conversion sequence for that parameter since it converts the implied object argument to the appropriate function pointer or reference required by that first parameter. — *end note*]

Change the note in 12.2.2.3 [over.match.oper]/3.4:

(3.4.5) [*Note*: A candidate synthesized from a member candidate has its ~~implicit~~ object parameter as the second parameter, thus implicit conversions are considered for the first, but not for the second, parameter. — *end note*]

Change the note in 12.2.2.5 [over.match.copy]/2:

2 In both cases, the argument list has one argument, which is the initializer expression. [*Note*: This argument will be compared against the first parameter of the constructors and against the ~~implicit~~ object parameter of the conversion functions. — end note]

Change the note in 12.2.2.6 [over.match.conv]/2:

2 The argument list has one argument, which is the initializer expression. [*Note*: This argument will be compared against the ~~implicit~~ object parameter of the conversion functions. — end note]

Change the note in 12.2.2.7 [over.match.ref]/2:

2 The argument list has one argument, which is the initializer expression. [*Note*: This argument will be compared against the ~~implicit~~ object parameter of the conversion functions. — end note]

Change 12.2.4.2 [over.best.ics]/4:

4 However, if the target is

(4.1) — the first parameter of a constructor or

(4.2) — the ~~implicit~~ object parameter of a user-defined conversion function

and the constructor or user-defined conversion function is a candidate by […]

Change 12.2.4.2 [over.best.ics]/7:

7 In all contexts, when converting to the implicit object parameter or when converting to the left operand of an assignment operation only standard conversion sequences are allowed. [*Note*: When converting to the explicit object parameter, if any, user-defined conversion sequences are allowed. - *end note*]

Change 12.2.4.2.3 [over.ics.user]/1:

1 If the user-defined conversion is specified by a conversion function, the initial standard conversion sequence converts the source type to the ~~implicit~~ object parameter of the conversion function.

Change 12.3 [over.over]/4:

4 Non-member functions ~~and~~, static member functions, and explicit object member functions match targets of function pointer type or reference to function type. ~~Non-static~~ Implicit object member functions match targets of pointer-to-member-function type.

[*Note 3*: If ~~a non-static~~ an implicit object member function is chosen, the result can be used only to form a pointer to member ([expr.unary.op]). — *end note*]

Change 12.4 [over.oper]/7:

7 An operator function shall either be a non-static member function or be a non-member function that has at least one non-object parameter whose type is a class, a reference to a class, an enumeration, or a reference to an enumeration.

Change 12.4.2 [over.unary]/1:

1 A *prefix unary operator function* is a function named `operator@` for a prefix *unary-operator* `@` ([expr.unary.op]) that is either a non-static member function ([class.mfct]) with no non-object parameters or a non-member function with one parameter.

Change 12.4.3 [over.binary]/1:

1 A *binary operator function* is a function named `operator@` for a binary operator `@` that is either a non-static member function ([class.mfct]) with one non-object parameter or a non-member function with two parameters.

Change 12.4.5 [over.sub]/1:

1 A *subscripting operator function* is a function named `operator[]` that is a non-static member function with exactly one non-object parameter.

Change 12.4.6 [over.ref]/1:

1 A *class member access operator function* is a function named `operator->` that is a non-static member function taking no non-object parameters.

Change 12.4.7 [over.inc]/1:

1 An *increment operator function* is a function named `operator++`. If this function is a non-static member function with no non-object parameters, or a non-member function with one parameter, it defines the prefix increment operator `++` for objects of that type. If the function is a non-static member function with one non-object parameter (which shall be of type `int`) or a non-member function with two parameters (the second of which shall be of type `int`), it defines the postfix increment operator `++` for objects of that type.

§ 9.2.6 Wording in 13 [temp]

In 13.8.3.3 [temp.dep.expr]/3, add a new kind of type dependence:

3 An *id-expression* is type-dependent if it is not a concept-id and it contains

(3.1) — an *identifier* associated by name lookup with one or more declarations declared with a dependent type,

(3.2) — an *identifier* associated by name lookup with a non-type *template-parameter* declared with a type that contains a placeholder type,

(3.3) — an *identifier* associated by name lookup with a variable declared with a type that contains a placeholder type ([dcl.spec.auto]) where the initializer is type-dependent,

(3.4) — an *identifier* associated by name lookup with one or more declarations of member functions of the current instantiation declared with a return type that contains a placeholder type,

(3.5) — an *identifier* associated by name lookup with a structured binding declaration whose *brace-or-equal-initializer* is type-dependent,

(3.5*) — an *identifier* associated by name lookup with an entity captured by copy ([expr.prim.lambda.capture]) in a *lambda-expression* that has an explicit object parameter whose type is dependent ([dcl.fct]),

(3.6) — the *identifier* `__func__` ([dcl.fct.def.general]), where any enclosing function is a template, a member of a class template, or a generic lambda,

(3.7) — a *template-id* that is dependent,

(3.8) — a *conversion-function-id* that specifies a dependent type, or

(3.9) — a *nested-name-specifier* or a *qualified-id* that names a member of an unknown specialization;

or if it names a dependent member of the current instantiation that is a static data member of type "array of unknown bound of T" for some T ([temp.static]).

§ 9.3 Feature-test macro [tab:cpp.predefined.ft]

Add to 15.11 [cpp.predefined]/table 17 ([tab:cpp.predefined.ft]):

`__cpp_explicit_this_parameter` with the appropriate value.

§ 10 Acknowledgements

The authors would like to thank:

— Jonathan Wakely, for bringing us all together by pointing out we were writing the same paper, twice

— Chandler Carruth for a lot of feedback and guidance around many design issues, but especially for help with use cases and the pointer-types for by-value passing

— Graham Heynes, Andrew Bennieston, Jeff Snyder for early feedback regarding the meaning of `this` inside function bodies

— Amy Worthington, Jackie Chen, Vittorio Romeo, Tristan Brindle, Agustín Bergé, Louis Dionne, and Michael Park for early feedback

— Jens Maurer, Richard Smith, Hubert Tong, Faisal Vali, and Daveed Vandevoorde for help with wording

— Ville Voutilainen, Herb Sutter, Titus Winters and Bjarne Stroustrup for their guidance in design-space exploration

— Eva Conti for furious copy editing, patience, and moral support

— Daveed Vandevoorde for his extensive feedback on recursive lambdas and implementation help

§ 11 References

[EffCpp] Scott Meyers. Effective C++, Third Edition.
https://www.aristeia.com/books.html

[P0798R0] Sy Brand. 2017-10-06. Monadic operations for std::optional.
https://wg21.link/p0798r0

[P0798R3] Sy Brand. 2019-01-21. Monadic operations for std::optional.
https://wg21.link/p0798r3

[P0826R0] Agustín Bergé. 2017-10-12. SFINAE-friendly std::bind.
https://wg21.link/p0826r0

[P0839R0] Richard Smith. 2017-10-16. Recursive Lambdas.
https://wg21.link/p0839r0

[P0847R0] Gašper Ažman, Sy Brand, Ben Deane, Barry Revzin. 2018-02-12. Deducing this.
https://wg21.link/p0847r0

[P0847R1] Gašper Ažman, Sy Brand, Ben Deane, Barry Revzin. 2018-10-07. Deducing this.
https://wg21.link/p0847r1

[P0847R2] Gašper Ažman, Sy Brand, Ben Deane, Barry Revzin. 2019-01-15. Deducing this.
https://wg21.link/p0847r2

[P0929R2] Jens Maurer. 2018-06-06. Checking for abstract class types.
https://wg21.link/p0929r2

[P1169R0] Barry Revzin, Casey Carter. 2018-10-07. static operator().
https://wg21.link/p1169r0

[P1221R1] Jason Rice. 2018-10-03. Parametric Expressions.
https://wg21.link/p1221r1

[P1787R6] S. Davis Herring. 2020-10-28. Declarations and where to find them.
https://wg21.link/p1787r6

[P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.
https://wg21.link/p2237r0